

Ranveer Thakur Chand

Patricia McManus

ITAI 1378

22 June 2026

### Reflective Journal

This workshop lab was my first time learning and implementing image classification using a neural network built in PyTorch. I had heard of PyTorch many times over the years when I did little projects with python, but this was my first time using it. This lab seemed easy at first because of the Chiwawa and muffin pictures, but it ended up being way more work than I thought it would be. By the end of this lab I had touched on data loading, augmentation, forward and backward passes, loss functions, optimizers, and the repeated error of overfitting.

The network we used was called MySkynet, a basic fully connected model since we couldn't use CNNs yet. Images got converted into tensors through a series of transforms, which then got split into training and validation sets, and then loaded into the model using PyTorch's DataLoader. The training loop ran the data through multiple epochs, calculated the error with a loss function, and used backpropagation to update the weights each pass. You could increase epochs instead of just training the model again and again but then you had the risk of overfitting which I encountered.

My first round of changes was straightforwardly simple. I bumped the epochs from 3 to 12 and dropped the learning rate from 0.1 to 0.05. That alone pushed validation accuracy from about 0.436 up to 0.9, which was a frankly really close to the score at the end I was able to achieve after hours of tweaking. Just giving the model more time to learn made a big difference but only for this first round.

From there I switched the optimizer from SGD to Adam with a learning rate of 0.001, based on advice I got from Gemini. Adam adapts its learning rate per parameter, which makes it more stable than vanilla SGD for this kind of task. After retraining a few times, the model started bottlenecking around 0.9333 validation accuracy.

Probably the most annoying part, but the simplest problem in the lab as well was when my visualization started showing negative percentages which I didn't notice until I trained the model 4 – 5 times. After debugging it, I found out that having F.softmax inside the forward pass was conflicting with nn.CrossEntropyLoss, which already handles that internally. Once I pulled softmax out of the forward method and only applied it after the model output for the visualization, it went back to the way it was. At least now I know to visualize the result after almost every change or training session.

I also dealt with overfitting a lot in this lab. In the start, it took me 4 sessions of 12 epochs to notice that the training accuracy was sitting at 1.0000 while validation was stuck at 0.8333; an obvious sign the model was memorizing rather than generalizing. I added data augmentation (random cropping, horizontal flips, color jitter) with the help of Gemini to the training transforms to give the model more varied examples. I had to dial back the crop scale after the augmentation was too aggressive and actually started hurting training accuracy. Then I

added Dropout layers after each hidden layer to further reduce the model's tendency to overfit. All this was done with the help of Gemini since I am still an intermediate programmer.

By the end, after a few more training runs, I settled at a best validation accuracy of 0.9333. I kept trying for 0.9667 but couldn't make the training session land on it, annoying but I know the fluctuation between runs is expected when using dropout with a small dataset. Gemini actually explained that as well: when validation sometimes outperforms training, the dropout is doing its job, not a sign something's overfitting or something being wrong.

In terms of real-world applications, what I realized is how the same pipeline labeled images, transforms, CNN-style classification, powers a lot of the computer vision tools I interact with daily. Medical imaging, defect detection in manufacturing, content moderation, and autonomous vehicles. Even something as small as a phone sorting your photo library by faces.

Overall, this lab had a lot of trial and error and I mean a lot. I was playing around with this lab for about 8 hours total, tweaking parameters, getting errors, fixing them, and running it all over again and again. It still sucked that I couldn't hit 100% accuracy even after everything me and Gemini did. But my guess is that it was probably just the limit of what MySkynet could do. Since we weren't allowed to use CNNs until the next lab, a basic fully-connected network can only do so much. Still, getting to 0.9333 was decent, technically my highest run was the 0.9667 but whatever.